

Agentic Engineering — Five Papers

A pragmatic reader's guide to the 26 May 2026 arXiv
digest

Compiled from arXiv preprints · 26 May 2026

May 2026

Chapter 0: The Work Moved Around the Model

Read these five papers back to back and a pattern surfaces that none of them states outright: almost none of the leverage is in the model anymore. Foundation Protocol wraps the protocols you already run without touching the agents underneath. MemForest beats systems six times its speed with an architecture change and the same backbones everyone else uses, and says so plainly — “the win lives in the structure of the memory, not in how clever you are at query time.” CUA-Gym trains a model ten times smaller to match a larger one's *untrained* baseline. ParaVT flips one inference-time switch — sequential to parallel — and beats its own base model with no retraining at all. Even QUEST, the one paper that is fundamentally about training, gets a 2-billion-parameter model into frontier deep-research territory not with scale but with better-manufactured data. Five teams, five subfields, one quiet consensus: the model is increasingly a fixed input, and the engineering that matters now lives in the layer wrapped around it.

That reframing organizes everything else in the bundle. Here are the threads worth carrying into the chapters.

The reward function is the hard part, and the new move is to manufacture it. QUEST and CUA-Gym were written by different groups about different domains — text research and GUI clicking — and arrived at the same conviction: you will run out of human-labeled data long before you run out of compute, so stop collecting tasks and start synthesizing them, with rewards a machine can check. QUEST’s rubric trees decompose “good answer” into a hierarchy of leaf criteria, some graded by a Python script, some by a judge comparing against a reference. CUA-Gym co-generates the task, the environment, and the reward function as one consistent tuple. The shared insight is that supervision is the bottleneck, not model capability, and that verifiable synthetic supervision is the way through. If you build agents, the rubric tree and the co-generated tuple are both worth stealing as evaluation harnesses even if you never run RL.

Don’t let one component both produce and grade. The most transferable engineering idea in the bundle shows up three times under three names. CUA-Gym puts an information barrier between the Generator that builds the golden state and the Discriminator that writes the reward, because an agent that grades its own construction degenerates into re-checking its own recipe. QUEST learned the hard way that scoring a report in isolation inflates every score toward perfect, and switched to judging against a reference. ParaVT’s parallel sub-agents outvote a bad crop because each works a disjoint window with no chain to propagate the error. Adversarial separation — producer apart from grader, peer apart from peer — is a cheap structural fix for the reward-hacking and error-cascading that otherwise poison a pipeline quietly enough that your eval still looks fine.

Parallelism is a design axis, not an optimization. Every paper here treats the serial pipeline as the enemy. ParaVT fans tool calls out in one turn so latency is bounded by the slowest call, not their sum. MemForest dissolves the serial LLM write-march into parallel chunk extraction and regenerates summaries lazily along only the paths a write touches. CUA-Gym builds resettable, state-injectable mock environments precisely so distributed RL workers can hammer a shared pool without colliding. QUEST’s context condenser is what lets a trajectory survive past the turn count it was trained on. The recurring shape is fan-out with compact, summarized returns — and it buys accuracy, bounded cost, and freshness at the same time.

Watch the process, not just the final answer. These papers keep their eyes on the part of the system everyone else ignores. MemForest’s headline discovery is that retrieval — the thing the field obsesses over — was a rounding error, and the real cost lived on the write path nobody budgets for. ParaVT’s failure mode only appears under temperature sampling during training, never under greedy decoding, so an eval can look healthy while training silently rots — the fix is to watch format compliance and tool-call rate *during* training. CUA-Gym’s whole reward design is about grounding the score in task completion rather than the construction recipe. The unit of attention has shifted from the output to the trajectory that produced it.

As models absorb capability, the constraint moves to coordination. Foundation Protocol names this directly — the bottleneck shifts from raw model capability to coordination, because in agent

systems communication *is* execution and the protocol layer becomes the safety boundary whether you designed it that way or not. ParaVT makes the same point from inside the model: RL has to cooperate with pretrained tool priors rather than override them, because the same prior that drives exploration also drives format collapse. The more the base model already knows how to do, the more the real work becomes managing what surrounds it — identity, budgets, memory, the mechanics of how and when it fires a tool.

Carry those five threads in as you read. The chapters run from the outside in: Foundation Protocol on coordinating a society of agents, QUEST and CUA-Gym on manufacturing the supervision that makes a single agent capable, ParaVT on the mechanics of how it fires tools in a single turn, and MemForest on what it remembers across thousands of them. Different layers of the same stack — and at every layer, the same lesson keeps reasserting itself: build the substrate right, and even the simple path reads it well.

Chapter 1: Foundation Protocol

Paper: [Foundation Protocol: A Coordination Layer for Agentic Society](#) · arXiv 2605.23218 · **Code:** github.com/FoundationAgents/foundation-protocol

Picture the workflow everyone keeps demoing: a personal agent books your travel across three vendors, haggles a refund, hires a price-monitoring agent and a payment agent, pauses to ask before spending real money, and settles up with receipts at the end. It's a great demo. It's also an integration nightmare. The tool calls go through MCP, the agent handoffs through A2A, the user approval through a UI delegation layer, the payment through a commerce protocol. Each carries its own idea of who's acting, what's authorized, and what got logged — and at every seam, identity drifts, the budget ceiling gets re-improvised, and the audit trail breaks. Stitch five protocols together and oversight becomes a pile of mismatched logs and prompt fragments.

That seam-stitching problem is what Foundation Protocol (FP) goes after. The pitch is deliberately narrow: FP is **not** an agent runtime, not an orchestrator, not yet another framework. It's a coordination layer — a control plane that sits across the protocols you already use and gives them one shared notion of entity, organization, value, and evidence. The bottleneck has shifted: as agents scale, the constraint stops being model capability and becomes coordination. Communication in agent systems *is* execution — a message can trigger code, spend money, delegate authority — so the protocol layer quietly becomes the safety boundary whether you designed it that way or not.

A graph, not a chat transcript

FP's core move is to treat everything as nodes and edges in one graph. Agents, tools, humans, services, institutions, and whole organizations are all **entities** — addressable, identity-bearing, accountable. Relationships, memberships, and sessions are edges; interactions are activities over that graph; policy and provenance sit on top as the evidence layer. The whole protocol reduces to seven nouns: Entity, Session, Activity, Envelope, Event, Receipt/Settlement, and Provenance — generic enough to express a single tool call, a multi-agent team, or a commercial transaction, but stable enough not to churn with every new interaction pattern.

A few primitives are where the engineering value lives. **Organizations are first-class entities** — they hold their own keys, sponsor sessions, carry budgets, and act as counterparties — so “this team requires human approval before deploying” becomes a rule attached to the org and enforced at a protocol boundary, not a convention buried in application code. A **session** binds participants, roles, policy references, and budget ceilings, so a bidder in an auction or a reviewer in a regulated pipeline is just a role. Collaboration emits **typed events** — an append-only stream agents read, operators inspect, and auditors replay — not one opaque chat log.

Then there's the economic spine, more concrete than most "agent economy" hand-waving. FP adds **metering, receipts, settlement references, and dispute signals** as ledger-agnostic primitives, so you can audit who delivered what and what it cost without FP mandating a payment rail. The reference implementation makes this real with a contract state machine, an arbiter that signs each transition into a hash chain, and two funding modes — escrow, where the protocol guarantees atomicity, or direct, where parties settle outside FP and the arbiter records a verifiable reference. Reputation is derived from closed-loop contract facts — rework counts, evidence completeness, settlement outcomes — not star ratings.

Why a control plane beats more point solutions

The deeper argument is institutional, and it's worth internalizing even if you never touch FP. When interoperability stays painful, the rational outcome is vertical integration: a few platforms swallow identity, routing, memory, policy, and settlement end to end, and the open agent ecosystem never materializes. When it's improvised instead, open networks form but stay fragile and hard to audit. A foundation layer is the bet that you don't have to pick. Two constraints carry it. First, **progressive disclosure** — entities advertise a short capability summary and reveal full schemas only after selection, a direct shot at the habit of stuffing giant tool descriptions into context before they're needed. Second, **keep the core small and push variability into profiles, extensions, and bridges**, so adoption is incremental rather than a flag-day migration. FP wraps rather than replaces — the reference stack ships an MCP bridge that turns any MCP server into an FP-addressable entity without modifying it, plus a CLI adapter that delegates agent work to Claude Code or Codex CLI through config, not code.

So what would you actually do with this? Most teams aren't building an AI economy tomorrow, and that's fine — FP is meant to be adopted from the edges inward. Start by wrapping a handful of tools or agents to get consistent identity, traces, and policy enforcement, then add organization and economic primitives as deployments mature. Even if you never adopt FP, the design is a checklist worth stealing. Audit your own multi-agent setup against its planes: is identity unified across your tool, agent, and UI layers, or re-invented at each? Are budgets and approvals enforced at a protocol boundary, or simulated in application logic an injected prompt could route around? Do access-control decisions leave traces, or vanish as silent drops? The most portable idea is the **checkpoint pipeline** — an ordered sequence of pluggable policy gates every inbound message traverses before reaching a handler. Human approval isn't bolted on top; it's a checkpoint with the same audit guarantees as the automated checks. If your system has nothing like that, you already know where the next incident is coming from.

Foundation Protocol is about coordinating many agents once they exist — the social and economic substrate a society of capable agents would need. But that presupposes the agents are capable in the first place, and frontier capability has to be trained in somehow. The next chapter flips the lens from coordination to capability, asking the harder upstream question: where does the training data for a genuinely competent deep-research agent come from when no human-labeled corpus is big or rich

enough? QUEST's answer is to stop collecting tasks and start manufacturing them — fully synthetic research problems built to make one agent formidable.

Chapter 2: QUEST

Paper: [QUEST: Training Frontier Deep Research Agents with Fully Synthetic Tasks](#) · arXiv 2605.24218 · **Code:** github.com/OSU-NLP-Group/QUEST

If you’ve ever looked at the deep-research agents shipping from the big labs and thought *I’d train my own, but I have no idea where the data comes from*, this is the paper that answers the question. The QUEST team built a family of open research agents — 2B all the way up to 35B parameters — that match or beat OpenAI’s and Google’s proprietary deep-research systems on several benchmarks, and they did it on **8,000 training tasks that no human ever wrote**. The whole supervision signal is synthetic. That’s the headline, and it’s the part worth stealing.

The trick is the rubric tree

The hard problem in training a research agent isn’t the model — it’s grading. A fact-seeking task like “which architect designed the house where Salinger lived for decades?” has one verifiable answer, so you can reward exact match. But “assess Apple’s entry into foldable phones” has no single right answer; quality lives in coverage, organization, sourcing, and insight. Most open agents pick one regime and overfit to it. Train only on single-answer questions and your agent gets good at trivia and bad at reports. That narrowness is exactly why existing open agents “generalize poorly across task types.”

QUEST’s move is to represent *every* task — objective or open-ended — as a **rubric tree**: a hierarchical breakdown of the constraints a good answer must satisfy. The root is the overall score. Internal nodes are higher-level criteria. Leaf nodes are directly checkable facts — “budget under \$200,” “material is wood,” “cites this source URL” — each scored binary by an automatic verifier. For open-ended tasks the top-level children are fixed (instruction-following, comprehensiveness, readability, insight) and the sub-criteria are generated per question. Because the tree encodes constraints rather than a unique answer, one structure covers tasks with a single answer, tasks with many valid answers, and fully open-ended ones. And because the root aggregates partial scores from its children, you get a graded reward — not the brittle 0/1 signal that wrecks credit assignment in reinforcement learning.

The synthesis pipeline is the practical part. They seed from trending Google keywords (keeping tasks topical and time-diverse), have a capable model — Claude Sonnet 4.5 — autonomously browse the web, derive verifiable constraints, and assemble a rubric tree. For objective tasks, GPT-5 then writes an executable **Python script** that grades each node programmatically. For open-ended tasks, a reference report is generated and a judge scores the candidate *against* that reference pairwise, because they learned the hard way that scoring a report in isolation just inflates everything toward a perfect mark. The filtering is brutal — 17,000 generated objective tasks become roughly 6,000 after

rubric refinement and verification — but the survivors come with reliable, machine-checkable rewards and zero annotation cost.

Three stages, three jobs, plus a memory that survives long horizons

QUEST trains in three stages, and the ablation makes the division of labor legible. **Mid-training** adapts the base model to long, noisy web interaction through two free byproduct tasks — summarizing accumulated context into a structured state, and extracting the relevant bits from raw HTML. **Supervised fine-tuning** teaches it to imitate good tool-use trajectories (distilled from a teacher, then rewritten with inline citations so every claim carries its source URL). **Reinforcement learning** — GRPO with the rubric-tree reward plus a fact-checking reward that re-fetches cited pages and verifies they actually support the claim — sharpens the open-ended report quality. Each stage earns its place: SFT lifts the objective benchmarks but drags down open-ended ones; mid-training generalizes across the board; RL recovers and pushes open-ended performance, at a small cost to pure reasoning tasks like HLE that read like an alignment tax. Full recipe wins overall.

Threaded through all of it is a **context condenser**. Instead of the usual discard-everything or keep-last-N truncation, QUEST compresses history into a structured JSON “Context State” with three buckets: *trusted* (verified facts with source URLs, reusable directly), *untrusted* (claims contradicted elsewhere, flagged with the reason), and *uncertain* (partially-supported claims with a concrete follow-up action attached). The agent resumes in a fresh window carrying forward what it knows, doubts, and still needs to check — so it doesn’t re-verify the same fact ten times. Training on session-level chunks between condensations let them fit 200K-token trajectories on 16 H100s and, neatly, let the agent extrapolate past 200 turns at inference despite being trained on at most 100.

What you’d actually take from this

The result that should reset your priors: even **QUEST-2B-SFT** hits 30.3 on HLE and 72.8 on GAIA, beating OpenAI’s o3 on those fact-seeking tasks. Strong synthetic data plus plain SFT gets a 2B model to credible deep-research territory — which is a real opening for privacy-sensitive, locally-hosted agents in healthcare or legal work, where you can’t ship queries to a frontier API. The catch: small models still fall apart on report synthesis, so the local-agent story holds for fact-finding, not yet for long-form writing.

The transferable instinct, even if you never train a model, is the rubric tree as an evaluation harness. If you’re building any agent whose output is hard to grade with a regex, decomposing “good” into a tree of individually-checkable leaf criteria — some verified by code, some by a judge comparing against a reference — gives you graded, automated feedback you can use for filtering, eval, or RL. The fact-checking reward is worth copying on its own: extract every cited claim, re-fetch the page, and verify support, capped so well-cited-but-wrong answers can’t game the score. And the honest

“unsuccessful attempts” section is a gift — pointwise scoring inflated, naive win/tie/lose collapsed, DPO on report pairs was too noisy. Those are exactly the dead ends you’d otherwise burn a month rediscovering. They released the models, data, and training scripts, so the whole recipe is auditable rather than aspirational.

The same instinct — manufacture the training signal so you don’t depend on scarce human labels, and make the reward verifiable rather than vibes — turns out to generalize past text research. The next chapter follows it onto the screen: CUA-Gym asks how you scale verifiable environments and reward functions for computer-use agents, where the “answer” isn’t a cited paragraph but a sequence of clicks that either did or didn’t accomplish the task.

Chapter 3: CUA-Gym

Paper: [CUA-Gym: Scaling Verifiable Training Environments and Tasks for Computer-Use Agents](#) · arXiv 2605.25624 · **Code:** github.com/xlang-ai/CUA-Gym

If you’ve tried reinforcement learning on a computer-use agent, you already know where the wall is. It isn’t the algorithm. It’s the data. Math RL has a clean recipe: generate a problem, check the answer, reward the policy. Code RL has the same shape: write a test, run it, score the result. But a single training instance for a computer-use agent (CUA) isn’t a question and an answer — it’s a *tuple* of three interlocking parts: a task instruction, an executable environment set to exactly the right starting state, and a reward function that can inspect the resulting screen-and-filesystem state and decide whether the agent actually succeeded. Hand-authoring one of those takes an expert hours, and every new application — Gmail, Notion, GIMP, a CRM — drags in its own setup quirks and verification surface. That’s why CUA RL datasets have stayed orders of magnitude smaller than their text-only cousins.

The useful move in CUA-Gym is treating that three-part tuple as something a team of coding agents can co-generate, not something humans must assemble. Hand it a topic specification and the pipeline produces all three components together, derived from one shared spec so they’re consistent by construction. The orchestration is the interesting part. A **Generator** agent reads the task and writes two scripts — `initial_setup.py` and `golden_patch.py` — which it runs on two separate VMs to materialize the starting state and the ideal “task-completed” golden state. A separate **Discriminator** agent writes the reward function. Crucially, the Discriminator sits behind a strict information barrier: it never sees the Generator’s setup scripts or working directory, only the task text and the two resulting environments. From those it decomposes the task into fine-grained sub-criteria — *emails exist in Sent, content matches the template, recipients match the Notion database* — and aggregates them into a progressive 0-to-1 score in `reward.py`. An **Orchestrator** drives the two agents through iterative rounds until the reward function reliably separates the initial state from the golden one under execution.

That information barrier is the load-bearing choice. If one agent both builds the golden state *and* grades it, the reward quietly degenerates into re-checking the construction recipe rather than measuring task completion — and a reward that just confirms its own setup gives a policy nothing to learn from. Splitting the roles adversarially forces the reward to be grounded in the task itself. The authors call it reward-hacking mitigated by construction, and their scaling curves bear it out: training stays monotonic, with none of the oscillation or reward-success decoupling that exploitable rewards usually trigger.

Loop-level agreement still can’t catch everything — ambiguous instructions, or tasks no policy could solve, only surface under rollout. So every accepted tuple passes a two-stage **filter**: an LLM majority vote scoring consistency, executability, hack-risk, clarity, and difficulty, then a teacher-model rollout

confirming the task is solvable and the reward tracks real success. That closes the gap between “internally consistent” and “actually trainable.”

The second axis is the one most teams underestimate: environments. You can’t train RL against the live web — auth, rate limits, and non-reproducible state make real sites impossible to inject, inspect, or reset. So the same agentic paradigm gets pointed at building environments. A Plan/Dev/Web agent trio synthesizes **CUA-Gym-Hub**, 94 self-contained mock web apps grounded in real software-usage distributions (from O*NET taxonomies and the Anthropic Economic Index), each a single-page app behind a unified HTTP API exposing session state for inspection, injection, and reset. State injection means one mock spawns many distinct tasks; session isolation means distributed RL workers hammer a shared pool without stepping on each other. The full dataset — CUA-Gym proper — is 32,112 verified tuples across 110 environments, the largest open CUA RL dataset to date and the only one combining programmatic verification with both desktop and web.

The results land where the engineering predicts. Trained with GSPO on Qwen3.5 MoE backbones, the A3B model jumps from 54.5 to 62.1 on OSWorld-Verified and the A17B from 62.2 to 72.6 — the smaller trained model matching the *untrained* larger base at roughly 10x fewer parameters, with gains holding at the bigger scale rather than plateauing. Both checkpoints also lift on the held-out WebArena browser benchmark, so the skill generalizes past the synthetic mocks. And one ablation worth internalizing: going from 10 environments to 80 at fixed data beat what more trajectories alone could recover. Environment *diversity* is a scaling axis in its own right — and one most pipelines don’t even instrument. There’s a charming emergent kicker, too: under fixed per-task step budgets, GSPO spontaneously taught the policy to pack multiple actions per turn, compressing trajectories 33–45% at matched performance.

So what does an engineer take from this? The portable idea isn’t the dataset — it’s the co-generation-with-an-information-barrier pattern. If you’re building verifiable rewards for *any* agentic domain where rewards are scarce, separate the agent that constructs the ground truth from the agent that grades it, give the grader only the task spec, and iterate until the reward discriminates. It’s a cheap structural fix for the reward-hacking that otherwise quietly poisons your training signal. The second borrowable lesson: invest in environment breadth, not just task count, and build environments as resettable, state-injectable mocks behind a state API so a few implementations fan out into thousands of isolated, parallel-safe training instances. Both moves are worth trying before you reach for a fancier optimizer — and the whole pipeline, dataset, environments, and models are being open-sourced, so you can fork it and point it at your own application stack.

Having seen how to manufacture the environments and rewards an agent learns *in*, the next chapter turns to the mechanics of how an agent actually fires its tools once it’s there. ParaVT tackles the “tool prior paradox” in parallel tool use — the awkward tension that shows up when you want an agent to call several tools at once during agentic video RL, and the model’s learned priors fight you. We move from generating the training substrate to tuning the RL machinery that runs on top of it.

Chapter 4: ParaVT

Paper: [ParaVT: Taming the Tool Prior Paradox for Parallel Tool Use in Agentic Video Reinforcement Learning](#) · arXiv 2605.20342 · **Code:** github.com/EvolvingLMs-Lab/ParaVT

If you have ever wired up an agent that calls the same tool three times in a row to gather evidence, ParaVT has a question for you: why are you doing it sequentially? Its core move is almost embarrassingly simple to state. Instead of a model that crops a chunk of video, reads it, crops another, reads that, and crops a third — each call re-inflating the context window and chaining its mistakes forward — ParaVT has the agent plan all the crops up front and fire them in **parallel** inside a single turn. Three independent sub-agents each grab their own time window, each returns a short text summary, and a main agent reasons over the bundle. The task is long-video question answering (“which player took the decisive volley in this ninety-minute match?”), but the lesson is not really about video.

Why parallel beats sequential

Sequential tool chains are brittle in three ways that any agent engineer will recognize. A single bad crop poisons everything downstream, because the next call reasons on top of the last one’s output with no peer to contradict it. The context window swells with every turn, since each tool response gets re-injected. And latency grows linearly — five calls cost five round-trips.

Parallel dispatch fixes all three at once. Because the sub-agents work on disjoint windows independently, a mis-localized crop gets **outvoted** by its correct peers rather than propagated down a chain. Because each sub-agent returns a compact summary instead of raw frames, context grows by a small constant rather than ballooning with visual tokens per turn. And because the calls run concurrently, total latency is bounded by the slowest sub-agent, not the sum — so dispatching *more* tools costs roughly the same as dispatching one. The paper makes a point of isolating this: take the same trained model and flip only the dispatch mode at inference, and parallel beats sequential on every benchmark, no retraining needed. That alone is worth stealing. If your harness already does multi-call evidence gathering, swapping serial-with-reinjection for parallel-with-summaries may buy you accuracy and a flatter cost curve for free.

The Tool Prior Paradox

The interesting trouble showed up when the team tried to *train* this behavior with reinforcement learning, and it generalizes well past video. Modern base models arrive with strong **pretrained tool priors** — they already “know,” from pretraining, how to emit tool calls. That prior is double-edged, and the paper names the trap the **Tool Prior Paradox**.

On the upside, you need that prior to get the model exploring tool use at all during RL. On the downside, two failures appear under temperature sampling. First, **format fragility**: under greedy decoding the model emits the clean `<think>/<tool_call>/<answer>` structure it was fine-tuned on, but a few RL steps in, the pretrained habit resurfaces — it reverts to the old `<tool_code>` tag, drops closing tags, stops emitting `<answer>`. Malformed rollouts can't be parsed into rewardable tool calls, so the whole RL advantage signal is computed over garbage. Second, the **skip-tool shortcut**: when overview frames already answer most questions, the reward gap between “called the tool” and “skipped the tool” is basically noise, so RL's group-normalized advantage on that axis is near zero and the policy learns the classic reward hack — stop calling tools entirely. Within seven steps, tool calls collapsed to nearly zero.

The team confirmed the diagnosis with a clean cross-model check. Run the identical setup on a weaker-prior model and the format stays perfect — but RL elicits *zero* tool calls. The same prior strength drives both the exploration you want and the format collapse you don't. You cannot simply pick a model with a tamer prior and call it a day.

The fix: anchor the format, gate the necessity

The remedy, **PARA-GRPO**, pairs one targeted mechanism with each failure. For format fragility, a **selective format reward** fires only at the specific structural-token positions where collapse concentrates — the closing tags — plus a small hook pinning the opening reasoning tag. Crucially it does not touch the content tokens, so it adds to the accuracy gradient instead of fighting it. For the skip-tool shortcut, **frame-budget randomization**: each prompt is given a random number of overview frames from a small set, so some fraction of prompts genuinely cannot be answered without cropping. On those, calling the tool measurably out-scores skipping it, manufacturing the reward contrast that vanilla RL had averaged away.

The ordering matters and the ablations show why neither half works alone. Anchoring without gating keeps rollouts well-formed but tool use stays lukewarm; gating without anchoring pushes tool calls up but leaves format broken, so the credit lands on unparseable junk. Only together do both axes stabilize. The headline numbers: training-time format compliance climbed from **0.13 to 0.64**, and the full model beat its Qwen3-VL base by **+7.9% on average** across six long-video benchmarks, even edging past GPT-4o on a couple of long-video splits.

What an engineer takes away

Two of these ideas travel far beyond cropping video. The first is the parallel-dispatch pattern itself: any time your agent gathers evidence from several independent sources — retrieval shards, file reads, sub-queries — consider fanning them out in one turn with summarized returns rather than chaining sequential calls that re-bloat context and amplify single failures. Peer-correctable evidence and bounded latency are general wins.

The second is the diagnosis. As tool use gets baked into base models, the priors they carry will both help and sabotage your fine-tuning, and the sabotage is sneaky — it only shows up under sampling, not under greedy decoding, so your eval may look fine while training quietly rots. If you RL-train tool use, watch format compliance and tool-call rate *during* training, not just final accuracy. And if a tool isn't strictly necessary on most of your prompts, expect the policy to learn to skip it; you may need to engineer scenarios where the tool genuinely earns its keep before the reward signal will ever reinforce it. The deeper point is that RL has to cooperate with pretrained priors rather than override them — a recipe the authors argue is transferable to any agentic setting where tools are optional, from retrieval-augmented generation to code execution.

That covers how an agent decides to fire its tools in the heat of a single turn. But firing tools well in the moment is only half of what makes an agent feel capable — the other half is what it remembers across hundreds of turns and long stretches of time. The next chapter turns to MemForest, an efficient agent memory system built around hierarchical temporal indexing, and the very different engineering problems that surface when the question shifts from “which tools do I call now?” to “what do I still know about what happened an hour, a day, or a thousand interactions ago?”

Chapter 5: MemForest

Paper: [MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing](#) · arXiv 2605.23986 · **Code:** github.com/Concyclics/MemForest

Here is a number worth pinning to the wall: when the MemForest team profiled where the time goes in long-running agent memory systems, retrieval — the part everyone obsesses over — was a rounding error. The real cost was on the *write* path. Systems like EverMemOS and MemoryOS spent hundreds, sometimes thousands, of seconds per question just turning fresh dialogue into usable memory. MemForest does the same job in roughly a sixth of the time, and the trick is almost entirely architectural rather than a smarter model.

The bottleneck nobody budgets for

If you have built an agent that remembers across sessions, you probably built it the way most systems do: new dialogue comes in, an LLM decides what to keep, then rewrites some compact “profile” or “summary” object to reconcile new facts with old. It feels clean. It is also quietly quadratic.

The paper names two structural sins. The first is putting the **LLM directly in the write critical path** — the model must synchronously adjudicate every chunk of new dialogue, forcing the pipeline into a serial march. The second is **full-state rewrites**: even when one trivial new fact arrives, the system regenerates the entire memory object. As memory grows, maintenance latency grows with it — you pay for the size of everything you have ever remembered, every time you remember one more thing.

There is a subtler problem too. Bob lived in Boston, moved to Davis, then to Miami. Ask “where does Bob live now?” and a compact profile answers fine: Miami. Ask “where did Bob live *before* Miami?” and the profile is useless — it compressed Davis away. Flatten everything into an embedding store instead and you hit the opposite failure: semantic similarity does not encode *time*, so the retriever hands back Boston or Miami because they look relevant, even though the answer is Davis. Both popular representations throw temporal structure away.

Three moves: parallelize, canonicalize, index by time

MemForest treats agent memory as a write-efficient temporal data-management problem — closer to a log-structured database index than a chatbot scratchpad. First, **parallel chunk extraction**: instead of one serialized LLM pass over a session, MemForest slices it into small fixed-size chunks (two turns by default) and extracts from them concurrently. The biggest latency source — that serial LLM march — dissolves into parallel calls.

Second, **canonical fact consolidation**. Parallel extraction produces overlapping candidates, so before indexing MemForest normalizes them into *canonical facts* — stable, time-anchored,

deduplicated units of memory. These are the durable source of truth, and critically they are mergeable: new evidence slots in without rewriting the scope’s entire history.

Third, the centerpiece — **MemTree**, a hierarchical temporal index that materializes each memory “scope” as a time-ordered tree. Leaves hold time-stamped evidence in chronological order; internal nodes summarize contiguous intervals; the root gives a coarse overview. MemForest builds three families: *session trees* preserving one conversation’s raw chronology, *entity trees* tracking an evolving subject like Bob’s residence, and *scene trees* grouping a topic across multiple entities.

Why does a time-ordered tree beat both the flat store and the rewritten profile? Order is explicit in the leaves, so predecessor and transition queries work — you browse to the interval just before Bob’s Miami move and recover Davis — and nothing gets overwritten when new evidence lands, since old states survive as leaves even as the root tracks the current trajectory. The payoff shows up at write time: a new fact becomes a leaf, and only the ancestors on its path to the root get marked “dirty.” Summaries regenerate *lazily* and in parallel, only for those dirty nodes, so the dependent work scales with the height of the tree — $O(\log N)$ — not the total memory size. MemForest does not make *everything* logarithmic; LLM extraction still depends on the incoming session. The honest claim is that post-extraction maintenance no longer blows up with the size of what you already remember. Retrieval mirrors this: a coarse **forest recall** picks relevant whole trees from their root summaries, then a **tree browse** descends from interval summaries to leaf evidence.

Reading the result

The numbers back the architecture. On LongMemEval-S, MemForest hits the best overall accuracy among stateful systems — 79.8% with the 30B backbone — while constructing memory about 6x faster than EverMemOS and up to 13.7x faster than MemoryOS. Its strongest categories are exactly the ones the design targets: multi-session and temporal-reasoning questions. On the harder LoCoMo benchmark it is competitive rather than dominant but stays balanced and keeps the write-path advantage. The trade-off is honest: MemForest spends *more* tokens than lean systems like Mem0, turning a few long prompts into many short parallel ones — it optimizes wall-clock freshness, not token count, and much of that overhead is repeated prefixes that prefix caching can claw back.

What should an engineer take from this? A reframing: if your agent’s memory is getting slow as it grows, the fix is probably not a better extraction prompt — it is getting the LLM off the synchronous write path and refusing to rewrite the whole state for every new fact. Two things to try without adopting the full system. One: **stop rewriting global profiles in place**. Append time-anchored facts as immutable units and regenerate summaries lazily, only along the paths a write touches. Two: **make your memory time-aware before you make it smarter**. If you lean on a flat vector store and see “wrong-time” answers — the latest or most similar fact instead of the correct historical one — that is the exact failure MemTree was built to kill, and a lightweight chronological index will help more than a bigger embedding model.

One last lesson sits in the ablations. The fanciest retrieval mode — an LLM-guided browse steered by a planner — gives the top score, but a cheap embedding-only descent over the *same* substrate gets most of the way there. That is the paper’s quiet thesis: the win lives in the structure of the memory, not in how clever you are at query time. Build the substrate right, and even the simple path reads it well.